

ARCTIC WOLF

Software Developer — First-Round Preparation Workbook

Java • DSA • Backend Engineering • Resume Deep Dive

ROUND FOCUS

The recruiter guidance says this is a hands-on technical assessment. Java is explicitly acceptable. Expect DSA/problem-solving, live Java coding, questions on frameworks and technologies you have worked with, project implementation/design decisions, and practical backend engineering concepts.

How to use this workbook

- Solve DSA problems in a blank Java editor and explain your reasoning aloud.
- Treat every resume metric and technology as an interview question.
- For project questions use: problem → design → implementation → trade-off → result.
- Practice failure scenarios, not only happy paths.

Prepared for: Sundeep Kumar Singh

28 September 2026

1. What This Round Is Testing

The new recruiter description makes this broader than a pure LeetCode screen. Prepare for both live coding and a deep technical discussion of your backend work.

Area	What to know / explain
DSA / problem solving	Clarify constraints, derive an efficient approach, code in Java, test edge cases and state complexity.
Java coding	Collections, strings, generics, exceptions, concurrency basics and clean object-oriented implementation.
Resume depth	Defend every important metric, architecture decision, technology choice and performance claim.
Backend engineering	REST, Spring Boot, persistence, transactions, caching, Kafka, concurrency, idempotency, retries and observability.
Project design	Explain why a component exists, alternatives considered and what happens when dependencies fail.
Debugging	Walk from symptom → evidence → hypothesis → experiment → fix → validation.

LIVE-CODING HABIT

Spend the first 1–3 minutes clarifying inputs, constraints and edge cases. State the approach and complexity before writing code.

2. Resume Deep-Dive Drill

Your resume covers Java/Spring Boot, Kafka, PostgreSQL, Kubernetes, Azure, Temporal, Redis, AI agents and distributed systems. The following are high-probability follow-up areas based on the resume itself.

Area	What to know / explain
5M+ events/day	How measured? Average vs peak? Kafka partitions/consumers? What happened during spikes?
~90% search latency reduction	Baseline? Slow query? Indexes? JPA Specifications? Materialized view? p95/p99 measurement?
~75% memory reduction	What caused OOM? Explain cursor pagination, streaming and selective parsing. Why did each help?
Temporal	Why Temporal? Workflow vs Activity? Retries, timeouts, idempotency, compensation and versioning?
Kafka Saga	Saga boundary? Exactly-once meaning? Duplicate events? Partial failure? Compensation?
Redis token bucket	Token generation, burst capacity, atomicity, TTL, distributed instances and Redis failure.
Kubernetes jobs	CronJob/ad-hoc design, parameterisation, duplicate execution, Helm templates and limits.
Observability	p50/p95/p99, logs/metrics/traces, correlation IDs and latency investigation.
AI log agent	LLM placement, sensitive data handling, evaluation, retries and model unavailability.
MCP / agents	Tools exposed, permissions, tool failures, retries and human approval.
Spring Boot	Configuration, validation, exception handling, authentication, retries, timeouts and testing.
PostgreSQL	Schema, indexes, joins, transactions, isolation, query plans and connection pooling.

METRIC RULE

For every percentage or throughput number prepare: baseline, measurement method, workload, change introduced, result and limitations.

3. P0 DSA — Master These in Java

Earlier interview reports include Longest Substring Without Repeating Characters and Next Greater Element. Those reports are examples, not guarantees. The list below is the core practice set.

#	Problem	Pattern	Target
1	Longest Substring Without Repeating Characters	Sliding window + HashMap	$O(n)$ / $O(k)$
2	Next Greater Element	Monotonic stack	$O(n)$ / $O(n)$
3	Rotate Image	Transpose + reverse	$O(n^2)$ / $O(1)$
4	Two Sum	HashMap complement	$O(n)$ / $O(n)$
5	3Sum	Sort + two pointers	$O(n^2)$
6	Subarray Sum Equals K	Prefix sum + map	$O(n)$ / $O(n)$
7	Binary Search	Search invariant	$O(\log n)$ / $O(1)$
8	Merge Intervals	Sort + merge	$O(n \log n)$
9	Kth Largest Element	Min-heap / Quickselect	$O(n \log k)$
10	Top K Frequent Elements	Map + heap/buckets	$O(n \log k)$
11	Binary Tree Level Order Traversal	BFS + queue	$O(n)$
12	Number of Islands	Grid DFS/BFS	$O(mn)$
13	Course Schedule	Topological sort	$O(V+E)$
14	Reverse Linked List	Pointer reversal	$O(n)$ / $O(1)$
15	Climbing Stairs	1D DP	$O(n)$ / $O(1)$
16	House Robber	Choose/skip DP	$O(n)$ / $O(1)$
17	Best Time to Buy/Sell Stock	Running minimum	$O(n)$ / $O(1)$
18	Valid Parentheses	Stack	$O(n)$ / $O(n)$
19	Group Anagrams	Canonical key + map	$O(n \cdot k \log k)$
20	Rotting Oranges	Multi-source BFS	$O(mn)$

For every problem: explain brute force → optimise → complexity → code → dry run → edge cases.

4. P1 DSA — Expand Coverage

#	Problem	Pattern	Target
1	Product of Array Except Self	Prefix/suffix	$O(n)$
2	Sort Colors	Dutch National Flag	$O(n) / O(1)$
3	Longest Consecutive Sequence	HashSet starts	$O(n)$ average
4	Find All Numbers Disappeared	Index marking	$O(n) / O(1)$
5	Minimum Window Substring	Sliding window	$O(n)$
6	Valid Anagram	Frequency counting	$O(n)$
7	Valid Palindrome	Two pointers	$O(n) / O(1)$
8	Linked List Cycle	Floyd pointers	$O(n) / O(1)$
9	Remove Nth Node From End	Two-pointer gap	$O(n) / O(1)$
10	Merge Two Sorted Lists	Two-pointer merge	$O(n+m)$
11	Maximum Depth of Binary Tree	DFS	$O(n)$
12	Validate BST	Range/in-order	$O(n)$
13	LCA of BST	BST property	$O(h)$
14	Path Sum	DFS target	$O(n)$
15	Clone Graph	DFS/BFS + map	$O(V+E)$
16	Connected Components	DFS/BFS/Union-Find	$O(V+E)$
17	Pacific Atlantic Water Flow	Reverse DFS/BFS	$O(mn)$
18	Merge K Sorted Lists	Min-heap	$O(N \log k)$
19	Meeting Rooms II	Sort + min-heap	$O(n \log n)$
20	Coin Change	Bottom-up DP	$O(\text{amount} \cdot \text{coins})$
21	Word Break	DP + dictionary	$O(n^2)$ typical
22	Longest Increasing Subsequence	DP / patience sorting	$O(n \log n)$
23	Subsets	Backtracking	$O(2^n)$
24	Combination Sum	Backtracking	Exponential
25	Search Rotated Sorted Array	Modified binary search	$O(\log n)$

5. Four High-Value Coding Patterns

Longest Substring Without Repeating Characters

Approach: Use a sliding window and a map of character $\rightarrow$ last index. Move left only forward using $\max(\text{left}, \text{lastSeen} + 1)$.

Trap: Do not move left backwards; clarify character assumptions.

Next Greater Element

Approach: Scan right-to-left with a decreasing monotonic stack. Pop elements that cannot be the next greater value.

Trap: Strictly greater vs greater-or-equal; empty stack means -1.

Rotate Image

Approach: Transpose the square matrix, then reverse every row. Both operations are in-place.

Trap: Confirm square input and carefully handle boundaries.

Luhn Algorithm

Approach: Process digits from the right; double every second digit, subtract 9 when needed, then validate $\text{sum} \% 10$.

Trap: Clarify accepted formatting and non-digit input.

LIVE-CODING TEMPLATE

Clarify $\rightarrow$ Example $\rightarrow$ Brute force $\rightarrow$ Optimise $\rightarrow$ Complexity $\rightarrow$ Code $\rightarrow$ Dry run $\rightarrow$ Edge cases $\rightarrow$ Follow-up.

6. Java Coding Fluency

Area	What to know / explain
Collections	ArrayList, HashMap, HashSet, TreeMap/TreeSet, PriorityQueue, ArrayDeque; know when each fits.
HashMap	equals/hashCode contract, collisions, resizing, load factor, mutable keys and complexity.
Strings	Immutability, StringBuilder, efficient concatenation and character handling.
Generics	Type parameters, wildcards, PECS and type erasure.
Streams	map/flatMap/filter/reduce/collect; know when a loop is clearer.
Exceptions	Checked vs unchecked, root-cause preservation and Spring exception handling.
Concurrency	synchronized, Lock, volatile, atomics, ConcurrentHashMap, races and deadlocks.
JVM	Heap, stack, metaspace, GC, class loading, JIT, memory leaks and OOM.
Testing	JUnit 5, Mockito, integration tests, Testcontainers and testing failure paths.
Modern Java	Records, Optional, switch expressions, CompletableFuture and Java 17/21 features.

Practice short implementations:

- LRU cache using HashMap + doubly linked list.
- Bounded blocking queue without ExecutorService.
- Token-bucket rate limiter.
- Top-K users using Java collections/streams.
- Thread-safe singleton with a clear explanation of safe publication.

7. Spring Boot & Backend Engineering

Area	What to know / explain
Request lifecycle	Controller → validation → service → repository → DB; transaction and exception boundaries.
Dependency injection	Constructor injection, bean lifecycle, scopes and circular dependencies.
Transactions	Propagation, isolation, rollback rules, lazy loading and self-invocation pitfalls.
JPA/Hibernate	N+1, fetch strategies, pagination, specifications, dirty checking and optimistic locking.
REST	Resource modelling, status codes, pagination, filtering, idempotency and versioning.
Resilience	Timeouts, retries/backoff, circuit breakers, bulkheads and safe fallback design.
Caching	Cache-aside, TTL, invalidation, stampede and Redis failure.
Security	Authentication vs authorisation, JWT, mTLS and secret handling.
Configuration	Profiles, environment variables, external configuration and safe defaults.
Testing	Unit vs integration, Testcontainers and contract testing.

Likely follow-ups:

- Why JPA Specifications for dynamic search?
- When would you use a materialized view?
- Offset vs cursor/keyset pagination at scale?
- How do you make a POST idempotent under retries?
- How do you diagnose a sudden p99 increase?
- What happens when the DB connection pool is exhausted?

8. Kafka & Distributed Systems

Area	What to know / explain
Partitions	Ordering is within a partition; explain key selection, hot partitions and consumer parallelism.
Consumer groups	Partition assignment, rebalancing and why consumers beyond partition count do not add parallelism.
Delivery semantics	At-most-once, at-least-once and exactly-once; distinguish broker guarantees from end-to-end correctness.
Idempotency	Stable event IDs/idempotency keys and durable deduplication/transactional boundaries.
Retries	Transient vs permanent failure, retry topics/DLQ, backoff and poison messages.
Saga	Orchestrated vs choreographed, compensation, partial failure and observability.
Ordering	Partition by entity key when per-entity ordering matters.
Backpressure	Bound work and downstream load; control consumer concurrency and batching.
Schema evolution	Compatibility, versioning and safe producer/consumer rollout.
Crash scenario	Consumer commits DB then crashes before acknowledgement: explain why duplicate processing can occur.

KEY DISTINCTION

Separate broker-level delivery guarantees from application-level correctness. At-least-once processing can still be correct when the downstream operation is idempotent.

9. PostgreSQL, SQL & Search

Area	What to know / explain
Indexes	B-tree, composite order, selectivity, covering indexes, write overhead and when indexes are ignored.
Query plans	EXPLAIN/EXPLAIN ANALYZE, scans, joins, estimated vs actual rows.
Joins	INNER/LEFT joins, cardinality, duplicate rows and predicate placement.
Transactions	ACID, isolation levels, locking and practical concurrency behaviour.
Pagination	Offset vs keyset/cursor pagination for large datasets.
Connection pools	Pool sizing, leaks, timeouts and application concurrency.
Materialized views	Precompute expensive reads; understand staleness and refresh cost.
Elasticsearch	Inverted index, mappings, analyzers, term vs match, filters and aggregations.
Query tuning	Measure first: plan, row counts, indexes, cardinality and workload.

Explain the ~90% latency improvement as an investigation, not a magic optimisation:

- Baseline and percentile selected.
- Slow query identified.
- Query plan inspected.
- Indexes/materialized view introduced.
- Generated SQL validated.
- Comparable workload used for before/after measurement.
- Freshness and regression risks considered.

10. Kubernetes, Cloud & Observability

Area	What to know / explain
Pod lifecycle	Startup, readiness/liveness, termination, CrashLoopBackOff and common failure modes.
Deployments	ReplicaSets, rolling updates, probes, resources and rollback.
Jobs/CronJobs	Scheduling, concurrency policy, retries and duplicate execution.
Config/secrets	ConfigMap vs Secret; never bake credentials into images.
Networking	Service discovery, DNS, ingress/gateway and network policies.
Helm	Templates, values and environment-specific configuration.
Observability	Logs, metrics and traces answer different questions; correlate with trace/request IDs.
Latency	p50/p95/p99 and why averages can hide tail behaviour.
JVM incidents	Use logs, GC metrics, thread dumps, heap dumps, resource limits and crash logs.

DEBUGGING DRILL

A Spring Boot pod crashes after deployment. Walk through image/version comparison, Kubernetes events, exit code, application logs, resource limits, JVM crash logs, agents/native libraries, probes and rollback criteria.

11. CodeForgeUI — Deep Dive

Your resume describes Java 21, Spring Boot 4, Spring AI, SSE, Kafka Saga orchestration, Redis token-bucket rate limiting, JWT, Eureka, Spring Cloud Config, Kubernetes live preview, Redis distributed locks, MinIO, Stripe webhooks and OpenTelemetry.

- Why SSE instead of WebSocket for AI code-generation output?
- How did you make generated-file persistence idempotent?
- Where is saga state stored and what happens after partial failure?
- How does the Redis token bucket work atomically?
- How do distributed locks prevent duplicate preview provisioning?
- Why pre-warmed Kubernetes pods and what is the cost trade-off?
- How does MinIO synchronise generated files with a running preview?
- How are dynamic subdomains routed to the correct preview?
- How are Stripe webhooks made idempotent?
- How are p50/p95/p99 measured and traces correlated?
- Why PostgreSQL + MongoDB + MinIO rather than one datastore?
- Where would you add backpressure for concurrent code-generation requests?

12. Mercedes me Charge — Deep Dive

This is production experience, so be prepared to explain architecture and engineering decisions without exposing confidential information.

- Draw the registration/contract lifecycle and identify service boundaries.
- Why multiple Spring Boot microservices and what does each own?
- How was EV customer registration data modelled?
- How did you handle API compatibility across EMEA, AMAP and China?
- How did Kafka fit the 5M+ events/day workload?
- How did you reduce search latency from ~1.2s to ~120ms?
- What did JPA Specifications solve and what SQL did they produce?
- Why use a materialized view and how is freshness handled?
- Explain cursor-based pagination and the memory reduction.
- Why Temporal and what reliability problems did it solve?
- What did mTLS protect?
- How did encrypted HTTP/Feign logging protect sensitive data?
- How did you test retries, duplicates and partial failures?
- Describe one production/debugging incident using evidence and a structured investigation.

CONFIDENTIALITY

Discuss architecture and engineering principles without disclosing internal hostnames, credentials, secrets, proprietary business rules or customer data.

13. AI / RAG / Agent Questions

Area	What to know / explain
RAG pipeline	Ingestion → chunking → embeddings → retrieval → prompt construction → generation → evaluation.
Retrieval quality	Chunking, metadata filters, hybrid search, reranking, top-k and retrieval evaluation.
Hallucination	Grounding, citations, constrained prompts, retrieval validation and evaluation datasets.
Agent design	Tools, orchestration, permissions, retries, timeouts, failures and human-in-the-loop.
MCP	What tools were exposed, how authorisation worked and what happened when a tool failed.
LLM failures	Timeouts, rate limits, malformed output, token limits and unavailable models.
Security	Prompt injection, sensitive-data leakage, tool abuse and least-privilege access.
Evaluation	Task success, retrieval quality, latency, cost and regression testing.

14. Three Hands-On Mock Rounds

Mock 1 — DSA, 45 min

10 min clarify + brute force; 25 min Java implementation; 5 min dry run; 5 min complexity/follow-up. Use Longest Substring + Next Greater Element.

Mock 2 — Java + DSA, 60 min

Top K Frequent Elements. Follow up with stream vs loop, memory constraints, bucket alternative and PriorityQueue internals.

Mock 3 — Backend + Resume, 60 min

Explain the 90% search-latency improvement, design the endpoint/query, discuss caching, Kafka retries/idempotency and diagnose p99 latency.

SELF-REVIEW

After every mock record: understanding, approach, code correctness, edge cases, complexity, Java fluency, communication, debugging depth and trade-offs.

15. 7-Day Emergency Plan

Day	Focus	Execution
Day 1	Java + arrays	HashMap/HashSet/PriorityQueue; Two Sum, 3Sum, Subarray Sum K.
Day 2	Sliding window + stack	Longest Substring, Minimum Window, Valid Parentheses, Next Greater Element.
Day 3	Trees + graphs	BFS/DFS, Islands, Course Schedule, Level Order, Validate BST.
Day 4	Backend	Spring transactions, JPA, pagination, indexes, REST, caching, retries, idempotency.
Day 5	Kafka + K8s	Partitions, consumer groups, delivery semantics, saga, Jobs/CronJobs, observability.
Day 6	Resume grilling	CodeForgeUI + meCharge; rehearse every metric and architecture decision.
Day 7	Full mock	45-min DSA + 45-min backend/resume mock. Review mistakes; avoid large new topics.

Daily rule

- Solve at least 2 problems without looking at a solution.
- Write Java from a blank editor and explain the solution aloud.
- Record one mistake and one follow-up question.
- Spend the final 20–30 minutes revising resume/project details.

16. Final Interview Checklist

Verify each statement before the interview:

- I can solve Longest Substring Without Repeating Characters in Java from scratch.
- I can implement Next Greater Element with a monotonic stack.
- I can explain sliding window, two pointers, prefix sum, BFS/DFS, heap and basic DP.
- I know HashMap, HashSet, PriorityQueue and ArrayDeque trade-offs.
- I can explain equals/hashCode and HashMap collision behaviour.
- I can explain @Transactional, isolation and common JPA performance problems.
- I can explain Kafka partitions, consumer groups, retries and idempotency.
- I can explain offset vs cursor/keyset pagination.
- I can read an EXPLAIN plan at a basic interview level.
- I can explain p50/p95/p99 and trace correlation.
- I can explain Kubernetes Deployment, Job, CronJob and probes.
- I can defend every metric on my resume.
- I can draw my main project architecture in under 3 minutes.
- I can explain at least one debugging/production incident.
- I can state trade-offs rather than claiming one technology is universally better.

FINAL MINDSET

The goal is not to show that you memorised solutions. Show that you can reason under constraints, write maintainable Java, test assumptions, debug systematically and explain the engineering trade-offs behind systems you have built.

Source basis

This workbook is based primarily on the recruiter guidance supplied for the current round, the provided Arctic Wolf job description, the provided resume, and the earlier interview-prep research workbook. Reported interview questions/processes are treated as examples, not guarantees.